Smart Tap Monitor – Build Guide

College IoT project · ESP32 · MQTT · Wokwi simulation + real hardware

1. What you are building

A device that sits on a household tap line and tells you, live, whether the tap is working and whether the water is safe. It is a small version of the idea behind SIH26255 (monitoring tap connections under Jal Jeevan Mission).

What it measures: water flow (litres per minute and total litres), water quality (TDS in ppm), and water temperature (also used to correct the TDS reading).

What it decides: one of four statuses.

Status	Meaning
WORKING	Valve is open, water is flowing, TDS is within the limit
NO SUPPLY	Valve is open but there has been no flow for the set time
POOR QUALITY	TDS is above 500 ppm (the valve closes automatically)
VALVE CLOSED	Valve was closed manually or by the auto shut-off

What it does with that: shows it on the OLED, publishes it to the internet over MQTT every 5 seconds, and lets you open or close the valve from your phone or laptop.

```
[Bucket A] --pump--> [YF-S201 flow sensor] --> [Solenoid valve] --> [Bucket
B]
  (TDS + temperature probes sit in Bucket A)

Sensors --> ESP32 --> OLED (local display)
                  |--> Wi-Fi --> MQTT broker --> phone / laptop dashboard
                  |--> Relay --> Solenoid valve (auto shut-off + remote
control)
```

2. Before you start

Your photo covers the main parts. These small extras make life easier, and all are cheap:

- A multimeter (to set the buck converter to 5 V safely).
- A spare 10 kΩ resistor (only needed if the flow sensor gives no pulses, see Troubleshooting).
- A 1N4007 diode (protects against the valve's voltage spike; optional but recommended).
- A measuring jug (to calibrate flow), and a tray or plastic sheet to keep water away from electronics.
- Hose clamps or waterproof tape for the pipe joints.

Safety rules

1. Keep the breadboard, ESP32, relay and the 12 V adapter plug well away from the buckets. Only the pump, valve, flow sensor and probes should touch water.
2. Wire everything with the 12 V adapter unplugged. Plug it in last.
3. Check polarity (+ and −) twice on the valve, pump and buck converter.
4. Do not run the pump dry, and do not leave it running for long with the valve closed.

3. The pin plan (same for Wokwi and real hardware)

ESP32 pin	Connects to
GPIO 27	Flow sensor signal (through the 10k + 20k divider on real hardware)
GPIO 34	TDS sensor analog output
GPIO 4	DS18B20 data (with 4.7k resistor to 3V3)
GPIO 26	Relay IN
GPIO 21 (SDA), GPIO 22 (SCL)	OLED
GPIO 35	Only in Wokwi: potentiometer standing in for the flow sensor
3V3	TDS sensor, DS18B20, OLED
VIN (5 V)	Flow sensor red wire, relay module VCC

ESP32 pin	Connects to
GND	Ground of everything

4. The code (one sketch for both Wokwi and real hardware)

Change `SIMULATE` to 1 for Wokwi and 0 for real hardware. Everything else stays the same. Also change `CHANGE_ME_123` in both topics to something unique (for example your name and a number), because the free MQTT broker is public.

```
// Smart Tap Monitor - ESP32
// SIMULATE 1 = Wokwi (potentiometers stand in for sensors)
// SIMULATE 0 = real hardware
#define SIMULATE 1

#include <WiFi.h>
#include <PubSubClient.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ---------- Settings you can change ----------
#if SIMULATE
const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_PASS = "";
#else
const char* WIFI_SSID = "YOUR_WIFI_NAME";
const char* WIFI_PASS = "YOUR_WIFI_PASSWORD";
#endif

const char* MQTT_HOST  = "broker.hivemq.com";
const char* TOPIC_DATA = "tapmon/CHANGE_ME_123/data";
const char* TOPIC_CMD  = "tapmon/CHANGE_ME_123/cmd";

const float TDS_LIMIT        = 500.0;   // ppm; above this = poor quality
const float TDS_CAL          = 1.0;     // calibration multiplier for TDS
const float PULSES_PER_LITRE = 450.0;   // YF-S201 typical value
const float FLOW_MIN         = 0.2;     // L/min; below this = no flow
const int   NO_SUPPLY_SECS   = 10;      // demo value (use 600 for real use)
```

```
const bool  AUTO_SHUTOFF     = true;    // close valve on poor quality
#define RELAY_ACTIVE_LOW 1              // set 0 if the valve acts inverted

// ---------- Pins ----------
const int PIN_FLOW     = 27;
const int PIN_TDS      = 34;
const int PIN_TEMP     = 4;
const int PIN_RELAY    = 26;
const int PIN_SIM_FLOW = 35;   // simulation only

// ---------- Objects and state ----------
OneWire oneWire(PIN_TEMP);
DallasTemperature tempSensor(&oneWire);
Adafruit_SSD1306 oled(128, 64, &Wire, -1);
WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);

volatile uint32_t pulseCount = 0;
void IRAM_ATTR onPulse() { pulseCount++; }

float flowLpm = 0, totalLitres = 0, tds = 0, tempC = 25;
bool valveOpen = true;
String status = "WORKING";
int noFlowSecs = 0;
unsigned long lastTick = 0, lastPublish = 0, lastWifiTry = 0, lastMqttTry =
0;

// ---------- Valve ----------
void setValve(bool open) {
  valveOpen = open;
  bool level = RELAY_ACTIVE_LOW ? !open : open;
  digitalWrite(PIN_RELAY, level ? HIGH : LOW);
}

// ---------- MQTT commands: send OPEN or CLOSE ----------
void onMessage(char* topic, byte* payload, unsigned int len) {
  String msg;
  for (unsigned int i = 0; i < len; i++) msg += (char)payload[i];
  msg.toUpperCase();
  if (msg == "OPEN") setValve(true);
  if (msg == "CLOSE") setValve(false);
}
```

```
// ---------- TDS ----------
float readTdsVoltage() {
  long sum = 0;
  for (int i = 0; i < 20; i++) { sum += analogRead(PIN_TDS); delay(5); }
  return (sum / 20.0) / 4095.0 * 3.3;
}

float voltageToTds(float v, float t) {
  float k = 1.0 + 0.02 * (t - 25.0);   // temperature compensation
  float vc = v / k;
  float ppm = (133.42 * vc * vc * vc - 255.86 * vc * vc + 857.39 * vc) * 0.5;
  return ppm * TDS_CAL;
}

// ---------- Status logic ----------
void updateStatus() {
  if (valveOpen && flowLpm < FLOW_MIN) noFlowSecs++; else noFlowSecs = 0;

  if (tds > TDS_LIMIT) {
    status = "POOR QUALITY";
    if (AUTO_SHUTOFF && valveOpen) setValve(false);
  } else if (!valveOpen) {
    status = "VALVE CLOSED";
  } else if (noFlowSecs >= NO_SUPPLY_SECS) {
    status = "NO SUPPLY";
  } else {
    status = "WORKING";
  }
}

// ---------- OLED ----------
void drawScreen() {
  oled.clearDisplay();
  oled.setTextSize(1);
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(0, 0);  oled.println(status);
  oled.setCursor(0, 11); oled.print("Flow: ");  oled.print(flowLpm, 1);
oled.println(" L/min");
  oled.setCursor(0, 22); oled.print("Total: "); oled.print(totalLitres, 1);
oled.println(" L");
  oled.setCursor(0, 33); oled.print("TDS: ");   oled.print(tds, 0);
oled.println(" ppm");
  oled.setCursor(0, 44); oled.print("Temp: ");  oled.print(tempC, 1);
```

```
oled.println(" C");
  oled.setCursor(0, 55); oled.print("Valve: "); oled.println(valveOpen ?
"OPEN" : "CLOSED");
  oled.display();
}

// ---------- Wi-Fi and MQTT (never blocks the sensors) ----------
void keepConnected() {
  if (WiFi.status() != WL_CONNECTED) {
    if (millis() - lastWifiTry > 10000) {
      lastWifiTry = millis();
      WiFi.begin(WIFI_SSID, WIFI_PASS);
    }
    return;
  }
  if (!mqtt.connected() && millis() - lastMqttTry > 5000) {
    lastMqttTry = millis();
    String id = "tapmon-" + String(random(0xffff), HEX);
    if (mqtt.connect(id.c_str())) mqtt.subscribe(TOPIC_CMD);
  }
}

void setup() {
  Serial.begin(115200);

  digitalWrite(PIN_RELAY, RELAY_ACTIVE_LOW ? HIGH : LOW);  // relay off at
boot
  pinMode(PIN_RELAY, OUTPUT);
  setValve(true);

  pinMode(PIN_TDS, INPUT);
  analogReadResolution(12);
  tempSensor.begin();

#if SIMULATE
  pinMode(PIN_SIM_FLOW, INPUT);
#else
  pinMode(PIN_FLOW, INPUT);
  attachInterrupt(digitalPinToInterrupt(PIN_FLOW), onPulse, FALLING);
#endif

  Wire.begin(21, 22);
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
```

```
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  mqtt.setServer(MQTT_HOST, 1883);
  mqtt.setCallback(onMessage);
  lastTick = millis();
}

void loop() {
  keepConnected();
  mqtt.loop();

  if (millis() - lastTick >= 1000) {
    float dt = (millis() - lastTick) / 1000.0;
    lastTick = millis();

#if SIMULATE
    flowLpm = analogRead(PIN_SIM_FLOW) / 4095.0 * 10.0;   // pot = 0 to 10
L/min
    totalLitres += flowLpm / 60.0 * dt;
#else
    noInterrupts();
    uint32_t p = pulseCount;
    pulseCount = 0;
    interrupts();
    float litres = p / PULSES_PER_LITRE;
    totalLitres += litres;
    flowLpm = litres / dt * 60.0;
#endif

    tempSensor.requestTemperatures();
    float t = tempSensor.getTempCByIndex(0);
    tempC = (t == DEVICE_DISCONNECTED_C) ? 25.0 : t;
    tds = voltageToTds(readTdsVoltage(), tempC);

    updateStatus();
    drawScreen();

    Serial.print(status);
    Serial.print(" | flow ");  Serial.print(flowLpm);
    Serial.print(" L/min | total "); Serial.print(totalLitres);
    Serial.print(" L | TDS ");  Serial.print(tds);
    Serial.print(" ppm | temp "); Serial.print(tempC);
```

```
    Serial.print(" C | valve "); Serial.println(valveOpen ? "OPEN" :
"CLOSED");
  }

  if (millis() - lastPublish >= 5000 && mqtt.connected()) {
    lastPublish = millis();
    char buf[200];
    snprintf(buf, sizeof(buf),
      R"
({"flow":%.2f,"litres":%.2f,"tds":%.0f,"temp":%.1f,"valve":"%s","status":"%s"
})",
      flowLpm, totalLitres, tds, tempC, valveOpen ? "OPEN" : "CLOSED",
status.c_str());
    mqtt.publish(TOPIC_DATA, buf);
  }
}
```

5. Part A – Build it in Wokwi first (about 30 to 45 minutes)

As far as I know, Wokwi has no YF-S201 or TDS sensor part, so two potentiometers (knobs) stand in for them. Everything else (DS18B20, OLED, relay, Wi-Fi, MQTT) works the same as on real hardware, so you test all your code and logic before touching a single wire.

Step 1. Create the project. Go to wokwi.com, sign in, click New Project, choose ESP32, then Arduino. Delete the default code and paste the sketch from Section 4 with `SIMULATE 1`.

Step 2. Add libraries. In the project there is a file called `libraries.txt` (create it if it is missing). Put these five lines in it:

```
OneWire
DallasTemperature
Adafruit GFX Library
Adafruit SSD1306
PubSubClient
```

Step 3. Add the parts. Click the blue + button and add: 1 × DS18B20, 1 × SSD1306 OLED, 1 × Relay Module, 2 × Potentiometer, 1 × LED, 1 × resistor (220 Ω) and 1 × resistor (4.7 kΩ).

Step 4. Wire them like this.

Part	Wire
Potentiometer 1 (acts as the TDS sensor)	VCC to 3V3, GND to GND, SIG to pin 34
Potentiometer 2 (acts as the flow sensor)	VCC to 3V3, GND to GND, SIG to pin 35
DS18B20	VCC to 3V3, GND to GND, DQ to pin 4. Put the 4.7 kΩ resistor between DQ and 3V3
OLED	VCC to 3V3, GND to GND, SDA to pin 21, SCL to pin 22
Relay module	VCC to VIN, GND to GND, IN to pin 26
Valve indicator	VIN to relay COM. Relay NO to LED anode. LED cathode through 220 Ω to GND

Step 5. Run it. Click the green Start button. The serial monitor should start printing a line every second, and the OLED should show the status.

Step 6. Try the scenarios.

- Turn the flow potentiometer up to around 50 percent. Status becomes WORKING and the flow reading rises.
- Turn it to zero and wait 10 seconds. Status becomes NO SUPPLY.
- Turn the TDS potentiometer past about 40 percent. TDS goes above 500 ppm, status becomes POOR QUALITY, and the relay clicks and the LED turns off (valve closed).
- Click the DS18B20 in the diagram and change its temperature to see the TDS value shift slightly.

Step 7. Check MQTT. Open the HiveMQ web client (search for HiveMQ websocket client), click Connect, and add a subscription to `tapmon/YOUR_TOPIC_NAME/#`. You should see a JSON message every 5 seconds. Publish `OPEN` or `CLOSE` to `tapmon/YOUR_TOPIC_NAME/cmd` and watch the relay react.

If the relay behaves backwards in Wokwi, change `RELAY_ACTIVE_LOW` to 0.

6. Part B – Build it on real hardware

Build and test one sensor at a time. After each step, upload the sketch with `SIMULATE 0` and watch the Serial Monitor (115200 baud). The sketch prints every value each second, so you can see whether each new part works before adding the next.

Step 1. Set up the Arduino IDE.

1. Install the Arduino IDE.
2. Open File, Preferences, and in Additional Boards Manager URLs paste: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
3. Open Boards Manager, search `esp32`, and install the one by Espressif Systems.
4. Select the board ESP32 Dev Module and the correct COM port. If no port appears, install the CP210x or CH340 USB driver (depends on your board).
5. Open Library Manager and install: OneWire, DallasTemperature, Adafruit GFX Library, Adafruit SSD1306, PubSubClient.
6. In the sketch set `SIMULATE 0`, enter your Wi-Fi name and password, and change `CHANGE_ME_123` in both topics. If upload gets stuck on Connecting, hold the BOOT button on the ESP32 until it starts.

Step 2. Place the ESP32 and make power rails. Put the ESP32 on the breadboard. Connect its 3V3 pin to one rail (call it the 3.3 V rail), its VIN pin to another (the 5 V rail), and GND to the ground rail. Power the ESP32 from your laptop over USB for now.

Step 3. DS18B20 temperature probe. Red wire to the 3.3 V rail, black to GND, yellow (or white) to GPIO 4. Put the 4.7 kΩ resistor between the data wire and the 3.3 V rail. Upload and check that Serial shows a sensible room temperature. If it shows 25 exactly, the probe is not detected (see Troubleshooting).

Step 4. OLED. VCC to 3.3 V, GND to GND, SDA to GPIO 21, SCL to GPIO 22. After upload the screen should show status and values.

Step 5. TDS sensor. Module VCC to 3.3 V, GND to GND, analog output (A) to GPIO 34. Powering it from 3.3 V keeps its output within what the ESP32 can read. Keep the small circuit board dry; only the probe goes in water. Test: probe in the tap water gives a few hundred ppm (it varies by city), probe in air gives close to zero, and a spoon of salt in a cup gives a very high reading.

Step 6. Flow sensor (needs the voltage divider). The YF-S201 runs on 5 V and its signal is 5 V, but the ESP32 pins only tolerate 3.3 V. Wire it like this:

- Red wire to the 5 V rail (VIN). Black wire to GND.

- Yellow signal wire to one end of the 10 kΩ resistor.
- The other end of the 10 kΩ goes to a free breadboard row. Connect that row to GPIO 27 and also to one end of the 20 kΩ resistor.
- The other end of the 20 kΩ goes to GND.

This divides 5 V down to about 3.3 V. Test: gently blow through the sensor in the direction of the arrow printed on it. The flow value on Serial should jump above zero.

Step 7. Relay and valve (adapter unplugged while wiring).

1. Relay module: VCC to 5 V rail, GND to GND, IN to GPIO 26. Reset the ESP32; the relay should click on (valve OPEN).
2. 12 V adapter plus (+) goes to the relay COM terminal.
3. Relay NO terminal goes to one wire of the valve (a solenoid valve has no polarity, but the wiring is simpler if you use the red or positive lead here).
4. The other valve wire goes to the adapter minus (−).
5. Optional but recommended: place a 1N4007 diode across the two valve wires, with the striped end (cathode) towards the plus side.
6. Plug in the adapter and test the valve with no water. You should hear it click open when the relay clicks on. If it is inverted, change `RELAY_ACTIVE_LOW`.

Step 8. Power for the finished project (optional). Using the USB cable from a laptop or power bank is simplest. If you want the 12 V adapter to power the ESP32 too, first adjust the buck converter with the multimeter until its output reads 5.0 V with nothing connected, and only then connect the output (+) to the ESP32 VIN pin and output (−) to GND. Never plug in USB at the same time as powering VIN.

The pump: check the voltage printed on it. If it is 12 V, power it directly from the adapter through a simple switch. Do not power the pump or the valve from ESP32 pins.

Step 9. Plumbing for the demo. Order along the pipe: Bucket A, pump, flow sensor, solenoid valve, Bucket B. Rules:

- Flow sensor and valve both have an arrow showing the flow direction. Point the arrows the way the water travels.
- Clamp or tape every joint, and put the buckets on a tray.
- Put the TDS probe and the DS18B20 probe in Bucket A (the supply). Adding a pinch of salt there is how you will demonstrate contamination.
- Some solenoid valves need some pressure to open. If the valve will not open with gravity alone, use the pump.

- When the valve auto-closes, switch the pump off by hand. Do not let it push against a closed valve.

Step 10. Dashboard. Use the HiveMQ web client on a laptop, or an MQTT dashboard app on your phone (for example IoT MQTT Panel on Android). Subscribe to your data topic, and publish `OPEN` or `CLOSE` to your cmd topic to control the valve. Remember the public broker is visible to anyone who guesses your topic, which is fine for a college demo but not for real use.

7. Demo script for your viva or presentation

Test	What you do	What should happen
Normal supply	Pump on, clean water, valve open	WORKING, flow and litres counting up
Supply failure	Switch the pump off for 10 seconds	NO SUPPLY on OLED and dashboard
Contamination	Stir a teaspoon of salt into Bucket A	TDS passes 500 ppm, POOR QUALITY, valve clicks shut
Remote control	Publish OPEN or CLOSE from your phone	Valve opens or closes
Recovery	Replace water, publish OPEN	WORKING again

8. Calibration (do this once)

Flow. Run the pump into the measuring jug until it holds exactly 1 litre, then compare with the Total value on the OLED. If the display says 0.8 L, set `PULSES_PER_LITRE` to 450 × 0.8 = 360. If you can, repeat with 2 litres and check.

TDS. The ESP32 analog input is slightly non-linear, so readings can be off. If you have a cheap TDS pen, measure the same water with both and set `TDS_CAL` to pen reading divided by your reading. If you have no pen, say in your report that the 500 ppm limit is approximate.

9. Troubleshooting

Problem	Likely cause and fix
Upload fails or says Connecting forever	Hold the BOOT button while uploading; check the USB cable carries data; install the driver
Temperature always 25.0	DS18B20 not detected: check the 4.7 kΩ resistor to 3.3 V and the data wire on GPIO 4
OLED stays blank	Swap SDA and SCL; confirm address 0x3C; check 3.3 V and GND
Flow stays 0 with water running	Check arrow direction and the divider. If still nothing, add a 10 kΩ pull-up between the yellow signal wire and 5 V (on the sensor side of the divider, never on the ESP32 pin)
Flow reads far too high or low	Calibrate `PULSES_PER_LITRE` as in Section 8
TDS jumps around	Keep wires short, keep the probe still and fully submerged, and keep the TDS board away from the relay and valve wires
Valve works backwards	Change `RELAY_ACTIVE_LOW` between 1 and 0
Valve will not open	Check 12 V reaches COM, and try the pump for pressure
ESP32 resets when valve or pump switches	Share the supply less: power ESP32 from USB or a separate buck output, and add the 1N4007 diode
No data on the dashboard	Check Wi-Fi name and password (2.4 GHz only), and that the topic names match exactly

10. Report and extension ideas

For your report, include the block diagram, the pin table, a photo of the circuit, serial or dashboard screenshots of each test in Section 7, and a short comparison with SIH26255 (which asks for village-level monitoring, ML-based alerts and syncing with government IDs; yours is the working prototype for a single tap).

Optional upgrades if you have time:

- Log the MQTT data to a CSV and run an anomaly detector (Isolation Forest) on flow and TDS in Python.
- Add Telegram alerts when the status changes.
- Build a small Streamlit dashboard that subscribes to the MQTT topic.
- Simulate several taps with a Python script publishing to different topics, to show village-level scaling.